

Yes. **Your main Bash/UNIX commands are enough as a starting point.** Don't spend a whole day learning UNIX now. For this D. E. Shaw role, UNIX is listed as an advantageous skill, while the bigger requirements are **computer fundamentals, problem solving, logical reasoning, scripting and communication.**

But add these quickly:

```
pwd, ls, cd, mkdir, cp, mv, rm
cat, head, tail, less
grep, find
chmod, chown
ps, top, kill
df, du
whoami, which, echo
```

Know what **processes, permissions, PATH, pipes, redirection and environment variables** mean. That's enough for now.

First: What does "Aptitude" mean?

When people say the assessment has **aptitude**, they usually mean timed questions testing your **logical and quantitative problem-solving ability**, rather than programming/CS knowledge.

Typical sections:

Quantitative aptitude

- Percentages
- Ratios
- Averages
- Profit/loss
- Time & work
- Time/speed/distance
- Probability
- Permutations/combinations
- Number systems
- Basic algebra

Logical reasoning

- Number/letter series
- Patterns
- Arrangements
- Logical deductions
- Puzzles
- Data interpretation

Verbal

- Reading comprehension
- Grammar
- Vocabulary
- Sentence completion

And potentially

- **Programming/DSA**
- CS fundamentals

The exact assessment format isn't established by the JD we have, so **don't assume it's only aptitude.**



Your 3-Day Plan

You have **28, 29, 30 Aug.**

The objective isn't:

"Finish everything."

It's:

Maximum breadth + enough depth to solve common questions + interview readiness.

I'd divide your preparation roughly:

DSA 25% → Core CS 30% → Aptitude 15% → Projects/programming 15% → Role-specific + interview 15%

DAY 1 — DSA + Programming + DBMS + Aptitude

1. DSA — ~3 hours

You already have ~40 LeetCode problems, so **don't start a new DSA course**.

Revise patterns:

Arrays / Strings

- Traversal
- Frequency maps
- Prefix sum
- Two pointers
- Sliding window

Searching

- Binary search
- Time complexity

Hashing

- `unordered_map`
- `unordered_set`
- Frequency/counting problems

Linked List

- Reverse
- Fast/slow pointers
- Cycle detection

Stack / Queue

- Basic implementation
- Parentheses
- Monotonic-stack intuition

Trees

- Traversals
- BST
- Height/depth

Graphs

- BFS
- DFS

Complexity

You MUST be comfortable saying:

$O(1)$

$O(\log n)$

$O(n)$

$O(n \log n)$

$O(n^2)$

and explaining **why**.

Do ~8–10 representative problems, not 30 random ones.

2. C/C++ — ~1.5 hours

This is important because you use C++ for DSA.

Revise:

- pointers

- references
- arrays
- strings
- functions
- recursion
- structs/classes
- stack vs heap
- pass by value/reference
- memory
- vector
- map
- unordered_map
- set
- stack
- queue
- iterators
- basic STL algorithms

Be able to explain:

| What's a pointer?

| Pointer vs reference?

| Stack vs heap?

| map vs unordered_map ?

| Array vs vector?

3. DBMS + SQL — ~2 hours

You're already strong here, so this should be **rapid revision**.

DBMS

Know:

- Primary key
- Foreign key
- Candidate key
- Normalization
- 1NF/2NF/3NF
- Index
- ACID
- Transactions
- Concurrency
- Deadlock

SQL

Practice writing:

SELECT
WHERE
GROUP BY
HAVING
ORDER BY
JOIN
LEFT JOIN
RIGHT JOIN
SUBQUERY

Do **5–8 SQL questions**.

Especially:

Find second-highest salary.

Find duplicate records.

Count employees by department.

Find employees without a department.

Difference between WHERE and HAVING.

4. Aptitude — ~1.5 hours

Don't watch 5-hour aptitude courses.

Do **timed practice**.

Prioritize:

1. Percentages
2. Ratios
3. Averages
4. Probability
5. Time/work
6. Speed/distance
7. Number systems
8. Permutations/combinations
9. Logical reasoning
10. Data interpretation

Your goal is learning **patterns + shortcuts**, not theory.

DAY 2 — OS + CN + OOP + Security + Aptitude

This is probably your **most important CS day**.

1. Operating Systems — ~2.5 hours

Know these extremely well:

Processes

- Process
- Process state
- PCB

- Context switching

Threads

- Process vs thread
- User vs kernel threads

Scheduling

- FCFS
- SJF
- SRTF
- Round Robin
- Priority

Understand the idea and be able to calculate basic scheduling questions.

Synchronization

- Race condition
- Critical section
- Mutex
- Semaphore

Deadlocks

Know:

4 necessary conditions

Mutual exclusion

Hold and wait

No preemption

Circular wait

Then:

- prevention
- avoidance
- detection

Memory

- Paging
- Virtual memory
- Page fault
- TLB
- Page replacement

Know the basic idea of:

- FIFO
- LRU

2. Computer Networks — ~2 hours

This is extremely high-value.

Know:

Models

Application
Transport
Network
Data Link
Physical

and the OSI model.

Protocols

- HTTP/HTTPS
- TCP
- UDP
- DNS
- DHCP
- ARP

- ICMP

TCP

Understand:

- 3-way handshake
- reliability
- flow control
- congestion concept

Networking

- IP address
- MAC address
- Port
- Router
- Switch
- Firewall
- LAN/WAN

DNS

You should be able to explain:

"What happens when I type [google.com](https://www.google.com) into my browser?"

This is a **very good interview question**.

3. OOP — ~1.5 hours

This comes up everywhere.

Know:

- Class
- Object
- Encapsulation

- Abstraction
- Inheritance
- Polymorphism

Especially:

Overloading vs overriding

Abstract class vs interface

Composition vs inheritance

Compile-time vs runtime polymorphism

4. Security — ~1 hour

You don't need cybersecurity depth.

Know:

- Authentication vs authorization
- Hashing
- Encryption
- Symmetric vs asymmetric encryption
- Digital signatures
- HTTPS/TLS
- Cookies
- JWT
- SQL injection
- XSS
- CSRF
- Password hashing
- Firewall

Your existing JWT project gives you a big advantage here.

5. Aptitude — ~1.5 hours

Another timed set.

Do not spend the entire day studying aptitude.

DAY 3 — MOCK + Projects + Role + Interview

This day is different.

Don't spend Day 3 consuming endless YouTube videos.

You need to start simulating the actual selection process.

Morning — Full Mock Assessment

Give yourself approximately:

2–2.5 hours

Mix:

- Aptitude
- DSA
- C/C++
- OOP
- DBMS
- SQL
- OS
- CN
- Computer fundamentals

Do it **timed**.

Then spend ~1.5 hours analyzing:

- What did I not know?
- What did I know but forget?
- What took too long?
- What mistakes were careless?

Then revise only those weaknesses.

Afternoon — YOUR PROJECT INTERVIEW

This is extremely important for you.

Your portfolio is already strong.

Pick your **movie application** and prepare it like an interviewer is trying to destroy it.

You should be able to explain:

Architecture

```
graph TD; User --> NextJS[Next.js frontend]; NextJS --> APIRoute[API route]; APIRoute --> Validation; Validation --> Service; Service --> Mongoose; Mongoose --> MongoDB;
```

User
↓
Next.js frontend
↓
API route
↓
Validation
↓
Service
↓
Mongoose
↓
MongoDB

Authentication

Be ready for:

How does login work?

How is JWT generated?

Where is it stored?

How do protected routes work?

What happens when the token expires?

How do you validate user input?

Why use Zod?

Why MongoDB?

Why Mongoose?

Why Next.js instead of Express?

How would you scale this?

What happens if MongoDB goes down?

How would you prevent unauthorized access to another user's watchlist?

You should be able to answer these **without opening your code**.

Your second project

Pick your strongest project after the movie app.

Prepare:

- Problem
- Architecture
- Technology choices
- Biggest difficulty
- What you learned
- What you'd improve
- Scalability

- Security

Role-specific preparation

The D. E. Shaw JD describes the Systems Helpdesk role as involving **enterprise/developer applications, infrastructure, technology support and security**, and specifically mentions UNIX, PowerShell and hardware.

So spend **1–1.5 hours** on:

UNIX

Already covered above.

Hardware

Know:

- CPU
- RAM
- ROM
- Cache
- SSD/HDD
- Motherboard
- GPU
- BIOS/UEFI
- PSU

Troubleshooting

Imagine:

"A user's computer is connected to Wi-Fi but can't access websites. How would you troubleshoot?"

Think:

Physical connection

↓

IP configuration

↓

Ping gateway

↓

Ping external IP

↓

DNS check

↓

Browser/application

↓

Firewall/proxy

That's the type of thinking useful for a systems-oriented role.



Final Evening — Interview Preparation

Prepare answers for these.

About you

Tell me about yourself.

Walk me through your resume.

Why D. E. Shaw?

Why this role?

Why should we hire you?

What are your strengths?

What's something you're currently improving?

Projects

Tell me about your best project.

- | What was your contribution?
- | Biggest technical challenge?
- | Why did you choose your stack?

CS

- | Process vs thread?
- | TCP vs UDP?
- | What happens when you enter a URL?
- | What is normalization?
- | What is an index?
- | Explain ACID.
- | What is polymorphism?
- | What is a deadlock?
- | What is virtual memory?

Behavioral

- | Tell me about a difficult problem you solved.
- | Tell me about a time you worked with someone else.
- | How do you learn a technology you don't know?
- | What happens when you're stuck?



What NOT to do

For these three days:

- ✗ Don't learn another framework.

- ✗ Don't start advanced PostgreSQL.
- ✗ Don't start advanced AI integration.
- ✗ Don't try to finish your entire BCA syllabus.
- ✗ Don't watch 10-hour playlists passively.
- ✗ Don't solve 50 random LeetCode questions.
- ✗ Don't spend 6 hours on UNIX.

Instead:

Learn → solve → recall → explain.

Your final priority hierarchy

If you somehow run out of time, use this order:

● MUST KNOW

1. **DSA**
2. **C/C++**
3. **DBMS + SQL**
4. **OS**
5. **Computer Networks**
6. **OOP**

● SHOULD KNOW

7. Computer fundamentals/hardware
8. Aptitude
9. Web/HTTP
10. Security
11. UNIX
12. Software Engineering

QUICK GRASP

- 13. Cloud
- 14. Python
- 15. AI/ML
- 16. Data mining

IGNORE

Research Methodology, Lean Startup, Mobile Programming, Environmental Studies, Yoga, etc.

One thing I'd change about your current approach

Don't wait until the 30th to start interview preparation.

From **tomorrow**, after every technical topic, spend 5 minutes answering:

"How would I explain this to an interviewer?"

Because you have a **1-hour interview on the same day as the assessment**. Your ability to communicate what you know matters just as much as knowing it—the JD explicitly lists communication skills alongside technical/problem-solving ability.

And honestly, with your **8.39 CGPA, top-15 ranking, full-stack projects and current DSA work**, you have a good base. These three days should be about **converting that base into assessment/interview readiness**, not trying to become a different candidate overnight.